

COMP111: Artificial Intelligence

Introduction to Tree Search Problems

Frank Wolter

Content

- ▶ Introduce **problem solving through tree search**;

Content

- ▶ Introduce **problem solving through tree search**;
- ▶ Show how such problems can be stated as **search graphs**;

Content

- ▶ Introduce **problem solving through tree search**;
- ▶ Show how such problems can be stated as **search graphs**;
- ▶ Show how the **search tree** is generated from the search graph;

Content

- ▶ Introduce **problem solving through tree search**;
- ▶ Show how such problems can be stated as **search graphs**;
- ▶ Show how the **search tree** is generated from the search graph;
- ▶ Introduce the main **blind** (also called **uninformed**) tree search algorithms:
 - ▶ breadth-first search (BFS);
 - ▶ depth-first search (DFS);

Content

- ▶ Introduce **problem solving through tree search**;
- ▶ Show how such problems can be stated as **search graphs**;
- ▶ Show how the **search tree** is generated from the search graph;
- ▶ Introduce the main **blind** (also called **uninformed**) tree search algorithms:
 - ▶ breadth-first search (BFS);
 - ▶ depth-first search (DFS);
- ▶ Introduce the main performance measures for tree search algorithms and analyse BFS and DFS.

Solving Problems by Searching

- ▶ Consider an agent with a **goal** and a set of **actions** it can perform to reach the goal (a **goal-based agent**).

Solving Problems by Searching

- ▶ Consider an agent with a **goal** and a set of **actions** it can perform to reach the goal (a **goal-based agent**).
- ▶ The goal is the state of affairs the agent wants to bring about.

Solving Problems by Searching

- ▶ Consider an agent with a **goal** and a set of **actions** it can perform to reach the goal (a **goal-based agent**).
- ▶ The goal is the state of affairs the agent wants to bring about.
- ▶ The difficulty is deciding **the order** in which the actions are carried out.

Solving Problems by Searching

- ▶ Consider an agent with a **goal** and a set of **actions** it can perform to reach the goal (a **goal-based agent**).
- ▶ The goal is the state of affairs the agent wants to bring about.
- ▶ The difficulty is deciding **the order** in which the actions are carried out.
- ▶ The agent thus wants to compute a **sequence of actions** leading to the goal, and possibly satisfying an optimality condition (e.g. being as cheap as possible).

Solving Problems by Searching

- ▶ Consider an agent with a **goal** and a set of **actions** it can perform to reach the goal (a **goal-based agent**).
- ▶ The goal is the state of affairs the agent wants to bring about.
- ▶ The difficulty is deciding **the order** in which the actions are carried out.
- ▶ The agent thus wants to compute a **sequence of actions** leading to the goal, and possibly satisfying an optimality condition (e.g. being as cheap as possible).
- ▶ Describing the **goal**, the relevant **states**, the possible **actions** and the **optimality criterion** is called **problem formulation**.

Solving Problems by Searching

- ▶ Consider an agent with a **goal** and a set of **actions** it can perform to reach the goal (a **goal-based agent**).
- ▶ The goal is the state of affairs the agent wants to bring about.
- ▶ The difficulty is deciding **the order** in which the actions are carried out.
- ▶ The agent thus wants to compute a **sequence of actions** leading to the goal, and possibly satisfying an optimality condition (e.g. being as cheap as possible).
- ▶ Describing the **goal**, the relevant **states**, the possible **actions** and the **optimality criterion** is called **problem formulation**.
- ▶ Looking for a sequence of actions that leads to a goal state (and is optimal) is called **search**.

Search is Everywhere

The algorithms in this lecture are 60 years old and still run, every day, inside:

- ▶ **Route planning:** Google Maps / satnav (search over a road graph);

Search is Everywhere

The algorithms in this lecture are 60 years old and still run, every day, inside:

- ▶ **Route planning:** Google Maps / satnav (search over a road graph);
- ▶ **Robotics and warehouses:** an Amazon or Ocado robot planning a collision-free path;

Search is Everywhere

The algorithms in this lecture are 60 years old and still run, every day, inside:

- ▶ **Route planning**: Google Maps / satnav (search over a road graph);
- ▶ **Robotics and warehouses**: an Amazon or Ocado robot planning a collision-free path;
- ▶ **Game AI**: from Deep Blue's chess search to AlphaGo's Monte-Carlo *tree* search;

Search is Everywhere

The algorithms in this lecture are 60 years old and still run, every day, inside:

- ▶ **Route planning**: Google Maps / satnav (search over a road graph);
- ▶ **Robotics and warehouses**: an Amazon or Ocado robot planning a collision-free path;
- ▶ **Game AI**: from Deep Blue's chess search to AlphaGo's Monte-Carlo *tree* search;
- ▶ **LLM agents** (2023–): a model that plans, calls a tool, observes and backtracks is exploring a tree of possible action sequences.

Search is Everywhere

The algorithms in this lecture are 60 years old and still run, every day, inside:

- ▶ **Route planning**: Google Maps / satnav (search over a road graph);
- ▶ **Robotics and warehouses**: an Amazon or Ocado robot planning a collision-free path;
- ▶ **Game AI**: from Deep Blue's chess search to AlphaGo's Monte-Carlo *tree* search;
- ▶ **LLM agents** (2023–): a model that plans, calls a tool, observes and backtracks is exploring a tree of possible action sequences.

The vocabulary we now define — **state**, **action**, **frontier**, **branching factor** — is the vocabulary used in all of these.

Four Running Examples

- ▶ Example 1: the 8-Puzzle
- ▶ Example 2: Holiday in Romania
- ▶ Example 3: the Vacuum cleaner world
- ▶ Example 4: the 8 Queens Problem

Four Running Examples

- ▶ Example 1: the 8-Puzzle
- ▶ Example 2: Holiday in Romania
- ▶ Example 3: the Vacuum cleaner world
- ▶ Example 4: the 8 Queens Problem

We use the same four examples throughout: first to **formulate** the problem, then to **measure** how hard search on it is.

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;
 - ▶ a **start state** s_{start} from S ;

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;
 - ▶ a **start state** s_{start} from S ;
 - ▶ a set $S_{\text{goal}} \subseteq S$ of **goal states**;

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;
 - ▶ a **start state** s_{start} from S ;
 - ▶ a set $S_{\text{goal}} \subseteq S$ of **goal states**;
 - ▶ a set A of **actions** taking the agent from one state in S to another; we often do not name the actions but simply say which states are reachable from a state by a single action;

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;
 - ▶ a **start state** s_{start} from S ;
 - ▶ a set $S_{\text{goal}} \subseteq S$ of **goal states**;
 - ▶ a set A of **actions** taking the agent from one state in S to another; we often do not name the actions but simply say which states are reachable from a state by a single action;
 - ▶ a **cost function** stating how expensive an action is. If all actions are equally expensive we set $\text{cost}(a) = 1$ for every action a .

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;
 - ▶ a **start state** s_{start} from S ;
 - ▶ a set $S_{\text{goal}} \subseteq S$ of **goal states**;
 - ▶ a set A of **actions** taking the agent from one state in S to another; we often do not name the actions but simply say which states are reachable from a state by a single action;
 - ▶ a **cost function** stating how expensive an action is. If all actions are equally expensive we set $\text{cost}(a) = 1$ for every action a .
- ▶ A **solution** is a sequence of actions which, performed in s_{start} , takes the agent to a state in S_{goal} . A solution is **optimal** if there is no cheaper solution.

Problem Formulation: the Search Graph

- ▶ First determine the set S of possible **states** of the problem.
- ▶ The **search graph** for the problem is then given by:
 - ▶ the set S of states;
 - ▶ a **start state** s_{start} from S ;
 - ▶ a set $S_{\text{goal}} \subseteq S$ of **goal states**;
 - ▶ a set A of **actions** taking the agent from one state in S to another; we often do not name the actions but simply say which states are reachable from a state by a single action;
 - ▶ a **cost function** stating how expensive an action is. If all actions are equally expensive we set $\text{cost}(a) = 1$ for every action a .
- ▶ A **solution** is a sequence of actions which, performed in s_{start} , takes the agent to a state in S_{goal} . A solution is **optimal** if there is no cheaper solution.

We now determine the search graph for each of the four problems.

Example 1: the 8-Puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Slide tiles into the empty square until the tiles are in order.

- States S : every way of filling the 3×3 grid with 1–8 and a blank;

Example 1: the 8-Puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Slide tiles into the empty square until the tiles are in order.

- ▶ States S : every way of filling the 3×3 grid with 1–8 and a blank;
- ▶ Start state s_{start} : a single state, e.g. the one shown on the left;

Example 1: the 8-Puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Slide tiles into the empty square until the tiles are in order.

- ▶ States S : every way of filling the 3×3 grid with 1–8 and a blank;
- ▶ Start state s_{start} : a single state, e.g. the one shown on the left;
- ▶ Goal states S_{goal} : the single state shown on the right;

Example 1: the 8-Puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Slide tiles into the empty square until the tiles are in order.

- ▶ States S : every way of filling the 3×3 grid with 1–8 and a blank;
- ▶ Start state s_{start} : a single state, e.g. the one shown on the left;
- ▶ Goal states S_{goal} : the single state shown on the right;
- ▶ Actions: slide the tile left of / right of / above / below the blank into the blank;

Example 1: the 8-Puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Slide tiles into the empty square until the tiles are in order.

- ▶ States S : every way of filling the 3×3 grid with 1–8 and a blank;
- ▶ Start state s_{start} : a single state, e.g. the one shown on the left;
- ▶ Goal states S_{goal} : the single state shown on the right;
- ▶ Actions: slide the tile left of / right of / above / below the blank into the blank;
- ▶ Cost: 1 per action, so an optimal solution is a **shortest** sequence of moves.

Exercise: drawing part of the search graph

- ▶ When drawing a search graph we abstract from the actions and simply draw an **arrow** from a state s to each state reachable from s by one action. These are the **successor states** of s . Conversely, s' is a **predecessor** of s if s is a successor of s' .

Exercise: drawing part of the search graph

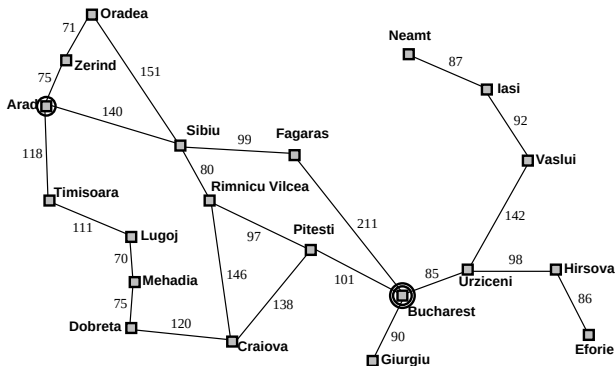
- ▶ When drawing a search graph we abstract from the actions and simply draw an **arrow** from a state s to each state reachable from s by one action. These are the **successor states** of s . Conversely, s' is a **predecessor** of s if s is a successor of s' .
- ▶ **Exercise.** Draw part of the search graph of the 8-puzzle: list the successor states of the start state on the previous slide, and the predecessor states of the goal state.

Exercise: drawing part of the search graph

- ▶ When drawing a search graph we abstract from the actions and simply draw an **arrow** from a state s to each state reachable from s by one action. These are the **successor states** of s . Conversely, s' is a **predecessor** of s if s is a successor of s' .
- ▶ **Exercise.** Draw part of the search graph of the 8-puzzle: list the successor states of the start state on the previous slide, and the predecessor states of the goal state.
- ▶ **Question.** The 8-puzzle graph is **undirected**: every action can be undone. Which of our four examples have this property, and which do not?

Example 2: Holiday in Romania

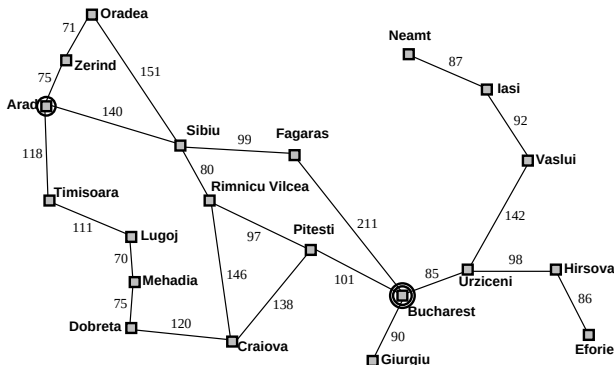
On holiday in Romania; currently in Arad. Drive as quickly as possible to Bucharest.



- States S : being in a city on the map: in Arad, in Zerind, in Oradea, ...

Example 2: Holiday in Romania

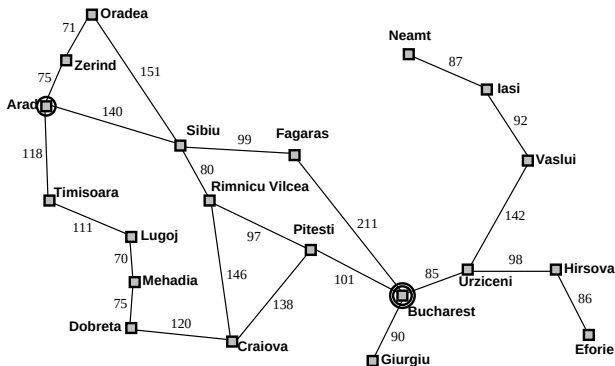
On holiday in Romania; currently in Arad. Drive as quickly as possible to Bucharest.



- States S : being in a city on the map: in Arad, in Zerind, in Oradea, ...
- Start state s_{start} : in Arad; Goal states S_{goal} : in Bucharest;

Example 2: Holiday in Romania

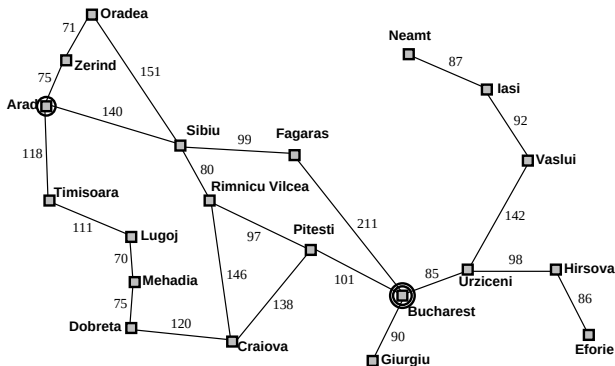
On holiday in Romania; currently in Arad. Drive as quickly as possible to Bucharest.



- ▶ States S : being in a city on the map: in Arad, in Zerind, in Oradea, ...
- ▶ Start state s_{start} : in Arad; Goal states S_{goal} : in Bucharest;
- ▶ Actions: drive from a to b for *neighbouring* cities a, b . There is no single action from Arad to Oradea;

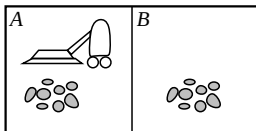
Example 2: Holiday in Romania

On holiday in Romania; currently in Arad. Drive as quickly as possible to Bucharest.

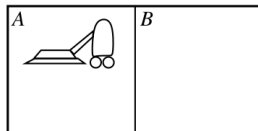


- ▶ States S : being in a city on the map: in Arad, in Zerind, in Oradea, ...
- ▶ Start state s_{start} : in Arad; Goal states S_{goal} : in Bucharest;
- ▶ Actions: drive from a to b for *neighbouring* cities a, b . There is no single action from Arad to Oradea;

Example 3: the Vacuum cleaner world



Possible start

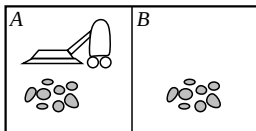


Possible goal

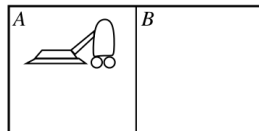
Two rooms; each may contain dirt; the cleaner is in one of them.

- States S : the 8 situations (cleaner left/right $\times$ dirt in each room: $2 \times 2 \times 2$);

Example 3: the Vacuum cleaner world



Possible start

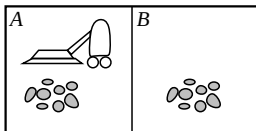


Possible goal

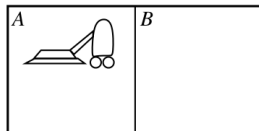
Two rooms; each may contain dirt; the cleaner is in one of them.

- ▶ States S : the 8 situations (cleaner left/right $\times$ dirt in each room: $2 \times 2 \times 2$);
- ▶ Start state s_{start} : any state; Goal states S_{goal} : both rooms clean;

Example 3: the Vacuum cleaner world



Possible start

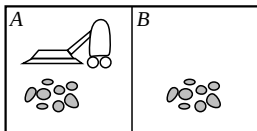


Possible goal

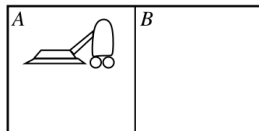
Two rooms; each may contain dirt; the cleaner is in one of them.

- ▶ States S : the 8 situations (cleaner left/right $\times$ dirt in each room: $2 \times 2 \times 2$);
- ▶ Start state s_{start} : any state; Goal states S_{goal} : both rooms clean;
- ▶ Actions: move left, move right, suck;

Example 3: the Vacuum cleaner world



Possible start

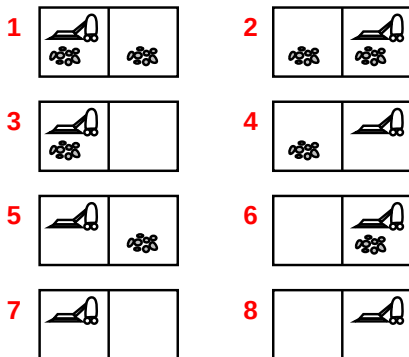


Possible goal

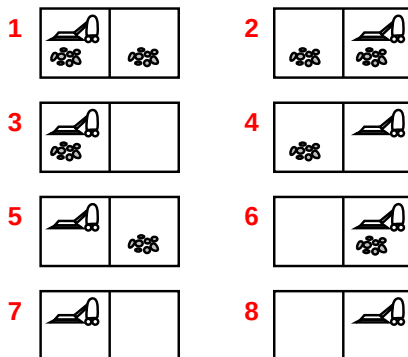
Two rooms; each may contain dirt; the cleaner is in one of them.

- ▶ States S : the 8 situations (cleaner left/right $\times$ dirt in each room: $2 \times 2 \times 2$);
- ▶ Start state s_{start} : any state; Goal states S_{goal} : both rooms clean;
- ▶ Actions: move left, move right, suck;
- ▶ Cost: 1 per action.

Example 3: the state space in full

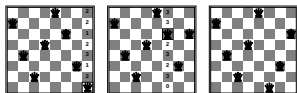


Example 3: the state space in full



This is the **whole** search graph — all 8 states and all transitions.

Example 4: the 8 Queens Problem

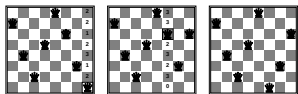


Place 8 queens on a chess board so that no queen attacks another.

(A queen attacks any piece in the same row, column or diagonal.)

- States S : any arrangement of 0 to 8 queens on the board;

Example 4: the 8 Queens Problem

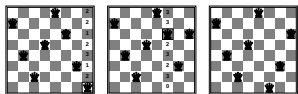


Place 8 queens on a chess board so that no queen attacks another.

(A queen attacks any piece in the same row, column or diagonal.)

- ▶ States S : any arrangement of 0 to 8 queens on the board;
- ▶ Start state s_{start} : the empty board;

Example 4: the 8 Queens Problem

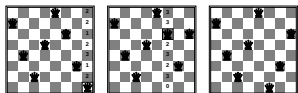


Place 8 queens on a chess board so that no queen attacks another.

(A queen attacks any piece in the same row, column or diagonal.)

- ▶ States S : any arrangement of 0 to 8 queens on the board;
- ▶ Start state s_{start} : the empty board;
- ▶ Goal states S_{goal} : 8 queens placed, no queen attacking another;

Example 4: the 8 Queens Problem

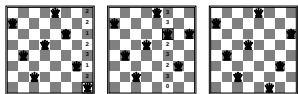


Place 8 queens on a chess board so that no queen attacks another.

(A queen attacks any piece in the same row, column or diagonal.)

- ▶ States S : any arrangement of 0 to 8 queens on the board;
- ▶ Start state s_{start} : the empty board;
- ▶ Goal states S_{goal} : 8 queens placed, no queen attacking another;
- ▶ Actions: add a queen to an empty square;

Example 4: the 8 Queens Problem



Place 8 queens on a chess board so that no queen attacks another.

(A queen attacks any piece in the same row, column or diagonal.)

- ▶ States S : any arrangement of 0 to 8 queens on the board;
- ▶ Start state s_{start} : the empty board;
- ▶ Goal states S_{goal} : 8 queens placed, no queen attacking another;
- ▶ Actions: add a queen to an empty square;
- ▶ Cost: irrelevant — **every** solution has the same length (8 actions). Here we only care *whether* we find a goal, not how we got there.

Formulation matters: two graphs for 8 Queens

Same problem, two search graphs:

Naive: add a queen to any empty square.

Better: add a queen to the leftmost empty column, on a square not attacked by an existing queen.

Formulation matters: two graphs for 8 Queens

Same problem, two search graphs:

Naive: add a queen to any empty square.

Better: add a queen to the leftmost empty column, on a square not attacked by an existing queen.

Does the better formulation lose solutions? No:

- ▶ every solution has one queen per column, so fixing the column order only removes "up to column permutation" equivalent paths.

Size of the search graph

Our examples are toys. Real state spaces are huge:

Problem	States	Brute-force search (10 million states/sec)
8-Puzzle	1.8×10^5	0.02 seconds
15-Puzzle	10^{13}	12 days
Rubik's Cube	4.3×10^{19}	140,000 years
24-Puzzle	10^{25}	24 billion years
Checkers	5×10^{20}	
Chess	10^{45}	
Go (19×19)	10^{170}	

Size of the search graph

Our examples are toys. Real state spaces are huge:

Problem	States	Brute-force search (10 million states/sec)
8-Puzzle	1.8×10^5	0.02 seconds
15-Puzzle	10^{13}	12 days
Rubik's Cube	4.3×10^{19}	140,000 years
24-Puzzle	10^{25}	24 billion years
Checkers	5×10^{20}	
Chess	10^{45}	
Go (19×19)	10^{170}	

For comparison: there are roughly 10^{80} atoms in the observable universe.

Size of the search graph

Our examples are toys. Real state spaces are huge:

Problem	States	Brute-force search (10 million states/sec)
8-Puzzle	1.8×10^5	0.02 seconds
15-Puzzle	10^{13}	12 days
Rubik's Cube	4.3×10^{19}	140,000 years
24-Puzzle	10^{25}	24 billion years
Checkers	5×10^{20}	
Chess	10^{45}	
Go (19×19)	10^{170}	

For comparison: there are roughly 10^{80} atoms in the observable universe.

From now on we assume the search graph cannot be stored. It is imaginary: we generate only the small part of it that we actually look at.

Size of the search graph

Our examples are toys. Real state spaces are huge:

Problem	States	Brute-force search (10 million states/sec)
8-Puzzle	10^6	0.01 seconds
15-Puzzle	10^{13}	6 days
Rubik's Cube	10^{18}	68,000 years
24-Puzzle	10^{26}	12 billion years
Checkers	10^{40}	(solved in 2007, after 18 years of computation)
Chess	10^{45}	
Go (19×19)	10^{170}	

Size of the search graph

Our examples are toys. Real state spaces are huge:

Problem	States	Brute-force search (10 million states/sec)
8-Puzzle	10^6	0.01 seconds
15-Puzzle	10^{13}	6 days
Rubik's Cube	10^{18}	68,000 years
24-Puzzle	10^{26}	12 billion years
Checkers	10^{40}	(solved in 2007, after 18 years of computation)
Chess	10^{45}	
Go (19×19)	10^{170}	

For comparison: there are about 10^{80} atoms in the observable universe.

Size of the search graph

Our examples are toys. Real state spaces are huge:

Problem	States	Brute-force search (10 million states/sec)
8-Puzzle	10^6	0.01 seconds
15-Puzzle	10^{13}	6 days
Rubik's Cube	10^{18}	68,000 years
24-Puzzle	10^{26}	12 billion years
Checkers	10^{40}	(solved in 2007, after 18 years of computation)
Chess	10^{45}	
Go (19×19)	10^{170}	

For comparison: there are about 10^{80} atoms in the observable universe. From now on we assume the search graph cannot be stored. It is imaginary: we only ever generate the small part of it that we look at.

Search

We assume that

- ▶ the **start state** s_{start} is given;

Search

We assume that

- ▶ the **start state** s_{start} is given;
- ▶ the actions are given in the form: *if in state s , then $s_1, \dots, s_n$ can be reached* (the **successor states** of s);

Search

We assume that

- ▶ the **start state** s_{start} is given;
- ▶ the actions are given in the form: *if in state s , then $s_1, \dots, s_n$ can be reached* (the **successor states** of s);
- ▶ a test for the **goal states** is given.

Search

We assume that

- ▶ the **start state** s_{start} is given;
- ▶ the actions are given in the form: *if in state s , then $s_1, \dots, s_n$ can be reached* (the **successor states** of s);
- ▶ a test for the **goal states** is given.

Search is the systematic exploration of the **search tree**: the tree of all **paths of states** that start at s_{start} and follow available actions, until a path to a goal state is found.

Search

We assume that

- ▶ the **start state** s_{start} is given;
- ▶ the actions are given in the form: *if in state s , then $s_1, \dots, s_n$ can be reached* (the **successor states** of s);
- ▶ a test for the **goal states** is given.

Search is the systematic exploration of the **search tree**: the tree of all **paths of states** that start at s_{start} and follow available actions, until a path to a goal state is found.

The search tree is also **imaginary**: it is generated as we go. Note it may be infinite even when the search *graph* is finite (paths may revisit states).

Search Tree for Holiday in Romania

The paths starting in Arad are:

- ▶ Arad;

Search Tree for Holiday in Romania

The paths starting in Arad are:

- ▶ Arad;
- ▶ Arad, Sibiu Arad, Timisoara Arad, Zerind;

Search Tree for Holiday in Romania

The paths starting in Arad are:

- ▶ Arad;
- ▶ Arad, Sibiu Arad, Timisoara Arad, Zerind;
- ▶ Arad, Sibiu, Arad Arad, Sibiu, Fagaras and so on.

Search Tree for Holiday in Romania

The paths starting in Arad are:

- ▶ Arad;
- ▶ Arad, Sibiu Arad, Timisoara Arad, Zerind;
- ▶ Arad, Sibiu, Arad Arad, Sibiu, Fagaras and so on.

So there is 1 path of length 0, 3 paths of length 1, 8 paths of length 2, ...

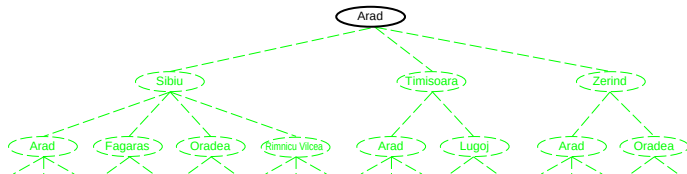
Search Tree for Holiday in Romania

The paths starting in Arad are:

- ▶ Arad;
- ▶ Arad, Sibiu Arad, Timisoara Arad, Zerind;
- ▶ Arad, Sibiu, Arad Arad, Sibiu, Fagaras and so on.

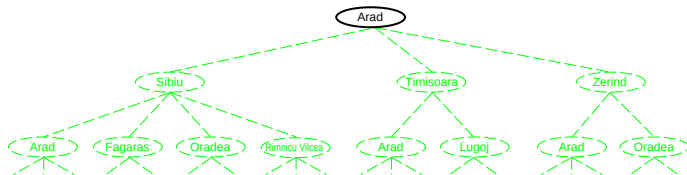
So there is 1 path of length 0, 3 paths of length 1, 8 paths of length 2, ... Note **Arad, Sibiu, Arad**: paths may revisit a state.

The tree is therefore infinite.



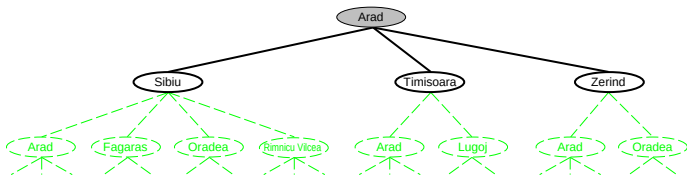
Growing the Search Tree

- ▶ The root of the tree is the start state s_{start} ;
- ▶ Applying actions, we grow longer paths by appending successor states to a path. This is called **expanding** the path.



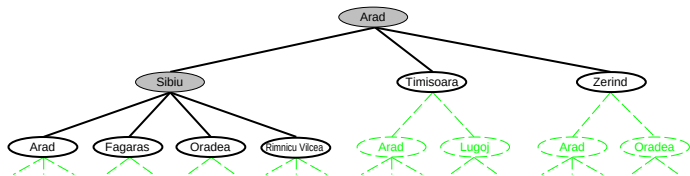
Growing the Search Tree

- ▶ The root of the tree is the start state s_{start} ;
- ▶ Applying actions, we grow longer paths by appending successor states to a path. This is called **expanding** the path.



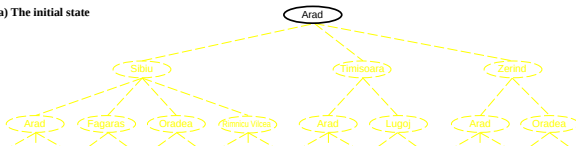
Growing the Search Tree

- ▶ The root of the tree is the start state s_{start} ;
- ▶ Applying actions, we grow longer paths by appending successor states to a path. This is called **expanding** the path.

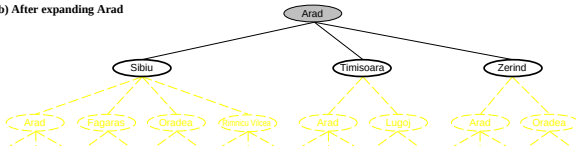


The Search Tree

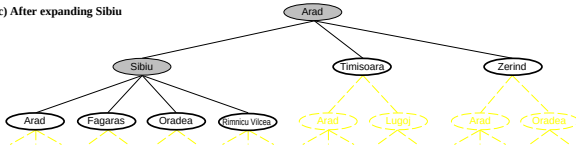
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



A search **algorithm** is a rule for deciding *which* path to grow next.

Pseudocode

In this module we give algorithms in **pseudocode**: informal, high-level descriptions intended for human rather than machine reading.

Pseudocode

In this module we give algorithms in **pseudocode**: informal, high-level descriptions intended for human rather than machine reading.

Pseudocode abstracts from implementation details and may leave steps (or subroutines) unspecified.

Pseudocode

In this module we give algorithms in **pseudocode**: informal, high-level descriptions intended for human rather than machine reading.

Pseudocode abstracts from implementation details and may leave steps (or subroutines) unspecified.

We now give a generic search algorithm which, in fact, leaves one very important step open.

Generic Algorithm for Search

- 1: **Input:** a start state s_0
- 2: for each state s the successors of s
- 3: a test $\text{goal}(s)$ checking whether s is a goal state
- 4: Set $\text{frontier} := \{s_0\}$
- 5: **while** frontier is not empty **do**
- 6: select and remove from frontier a path $s_0 \dots s_k$
- 7: **if** $\text{goal}(s_k)$ **then**
- 8: return $s_0 \dots s_k$ (and terminate)
- 9: **else** for every successor s of s_k add $s_0 \dots s_k s$ to frontier
- 10: **end if**
- 11: **end while**
- 12: return failure

Generic Algorithm for Search

```
1: Input: a start state  $s_0$ 
2:   for each state  $s$  the successors of  $s$ 
3:   a test  $\text{goal}(s)$  checking whether  $s$  is a goal state
4: Set  $\text{frontier} := \{s_0\}$ 
5: while  $\text{frontier}$  is not empty do
6:   select and remove from  $\text{frontier}$  a path  $s_0 \dots s_k$ 
7:   if  $\text{goal}(s_k)$  then
8:     return  $s_0 \dots s_k$  (and terminate)
9:   else for every successor  $s$  of  $s_k$  add  $s_0 \dots s_k s$  to  $\text{frontier}$ 
10:  end if
11: end while
12: return failure
```

The **frontier** (also called **agenda**) holds the paths generated but not yet expanded.

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.
- ▶ The algorithm maintains a set frontier of **paths**, initially just the path s_0 .

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.
- ▶ The algorithm maintains a set frontier of **paths**, initially just the path s_0 .
- ▶ While frontier is non-empty, it **selects and removes** a path from it (we say the path is **expanded**).

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.
- ▶ The algorithm maintains a set frontier of **paths**, initially just the path s_0 .
- ▶ While frontier is non-empty, it **selects and removes** a path from it (we say the path is **expanded**).
- ▶ If the last state of that path is a goal state, it terminates and returns the path.

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.
- ▶ The algorithm maintains a set frontier of **paths**, initially just the path s_0 .
- ▶ While frontier is non-empty, it **selects and removes** a path from it (we say the path is **expanded**).
- ▶ If the last state of that path is a goal state, it terminates and returns the path.
- ▶ Otherwise it adds to frontier every path obtained by appending one successor.

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.
- ▶ The algorithm maintains a set frontier of **paths**, initially just the path s_0 .
- ▶ While frontier is non-empty, it **selects and removes** a path from it (we say the path is **expanded**).
- ▶ If the last state of that path is a goal state, it terminates and returns the path.
- ▶ Otherwise it adds to frontier every path obtained by appending one successor.
- ▶ Then it repeats, selecting from the **updated** frontier.

Generic Algorithm for Search: comments

- ▶ The input is a start state s_0 , a way of producing the successors of a state, and a way of testing whether a state is a goal state.
- ▶ The algorithm maintains a set frontier of **paths**, initially just the path s_0 .
- ▶ While frontier is non-empty, it **selects and removes** a path from it (we say the path is **expanded**).
- ▶ If the last state of that path is a goal state, it terminates and returns the path.
- ▶ Otherwise it adds to frontier every path obtained by appending one successor.
- ▶ Then it repeats, selecting from the **updated** frontier.
- ▶ Note: the frontier stores **paths**, not states — we must return the route, not just the news that a goal exists.

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier?

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier? Equivalently: which path do we **expand**? (Or loosely: which state s_k do we expand?)

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier? Equivalently: which path do we **expand**? (Or loosely: which state s_k do we expand?)
- ▶ In which order do we add the new paths $s_0 \dots s_k s$ to the frontier?

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier? Equivalently: which path do we **expand**? (Or loosely: which state s_k do we expand?)
- ▶ In which order do we add the new paths $s_0 \dots s_k s$ to the frontier?

The second question we ignore (**don't-care non-determinism**). The first question **is** the algorithm:

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier? Equivalently: which path do we **expand**? (Or loosely: which state s_k do we expand?)
- ▶ In which order do we add the new paths $s_0 \dots s_k s$ to the frontier?

The second question we ignore (**don't-care non-determinism**). The first question **is** the algorithm:

- ▶ **Breadth-first search (BFS)**: always select the path **first** added to the frontier (frontier as a **queue**, FIFO);

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier? Equivalently: which path do we **expand**? (Or loosely: which state s_k do we expand?)
- ▶ In which order do we add the new paths $s_0 \dots s_k s$ to the frontier?

The second question we ignore (**don't-care non-determinism**). The first question **is** the algorithm:

- ▶ **Breadth-first search (BFS)**: always select the path **first** added to the frontier (frontier as a **queue**, FIFO);
- ▶ **Depth-first search (DFS)**: always select the path **last** added to the frontier (frontier as a **stack**, LIFO).

What the Generic Algorithm Leaves Open

- ▶ Which path $s_0 \dots s_k$ do we select and remove from the frontier? Equivalently: which path do we **expand**? (Or loosely: which state s_k do we expand?)
- ▶ In which order do we add the new paths $s_0 \dots s_k$ s to the frontier?

The second question we ignore (**don't-care non-determinism**). The first question **is** the algorithm:

- ▶ **Breadth-first search (BFS)**: always select the path **first** added to the frontier (frontier as a **queue**, FIFO);
- ▶ **Depth-first search (DFS)**: always select the path **last** added to the frontier (frontier as a **stack**, LIFO).

One line of difference. Completely different behaviour.

Breadth-First Search (BFS)

- ▶ Start by expanding the start state — this gives the tree down to depth 1;

Breadth-First Search (BFS)

- ▶ Start by expanding the start state — this gives the tree down to depth 1;
- ▶ Then expand **all** paths produced in the previous step — depth 2;

Breadth-First Search (BFS)

- ▶ Start by expanding the start state — this gives the tree down to depth 1;
- ▶ Then expand **all** paths produced in the previous step — depth 2;
- ▶ Then expand **all** paths produced in that step — depth 3;

Breadth-First Search (BFS)

- ▶ Start by expanding the start state — this gives the tree down to depth 1;
- ▶ Then expand **all** paths produced in the previous step — depth 2;
- ▶ Then expand **all** paths produced in that step — depth 3;
- ▶ and so on.

Breadth-First Search (BFS)

- ▶ Start by expanding the start state — this gives the tree down to depth 1;
- ▶ Then expand **all** paths produced in the previous step — depth 2;
- ▶ Then expand **all** paths produced in that step — depth 3;
- ▶ and so on.

In general: expand all paths of length n before any path of length $n + 1$.

Breadth-First Search (BFS)

- ▶ Start by expanding the start state — this gives the tree down to depth 1;
- ▶ Then expand **all** paths produced in the previous step — depth 2;
- ▶ Then expand **all** paths produced in that step — depth 3;
- ▶ and so on.

In general: expand all paths of length n before any path of length $n + 1$.

Implementation: the frontier is a **FIFO queue** — new paths go to the back.

Breadth-First Search: pseudocode

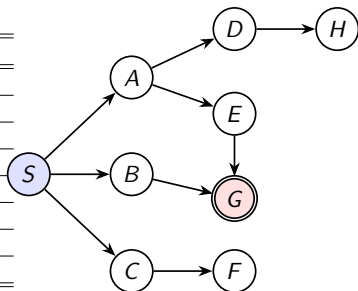
For the **select** step, always select the path **first added** to the frontier:

```
1: Input: a start state  $s_0$ 
2:   for each state  $s$  the successors of  $s$ 
3:   a test  $\text{goal}(s)$  checking whether  $s$  is a goal state
4:
5: Set  $\text{frontier} := \{s_0\}$ 
6: while frontier is not empty do
7:   select and remove from frontier the path  $s_0 \dots s_k$  that was
8:   first added to frontier
9:   if  $\text{goal}(s_k)$  then
10:    return  $s_0 \dots s_k$  (and terminate)
11:   else for every successor  $s$  of  $s_k$  add  $s_0 \dots s_k s$  to frontier
12:   end if
13: end while
```

Breadth-First Search: worked example

Reaching G from S :

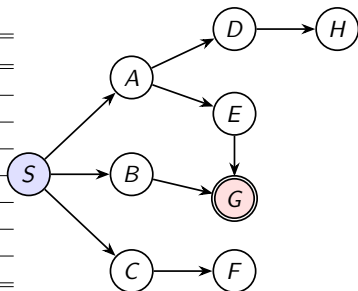
Expanded path	Frontier
	$\{S\}$



Breadth-First Search: worked example

Reaching G from S :

Expanded path	Frontier
	$\{S\}$
S not goal	$\{SA, SB, SC\}$

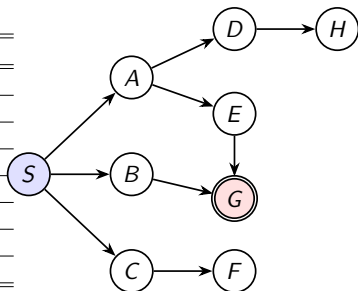


Selected: S . Goal? No. Expand: add SA, SB, SC .

Breadth-First Search: worked example

Reaching G from S :

Expanded path	Frontier
	$\{S\}$
S not goal	$\{SA, SB, SC\}$
SA not goal	$\{SB, SC, SAD, SAE\}$

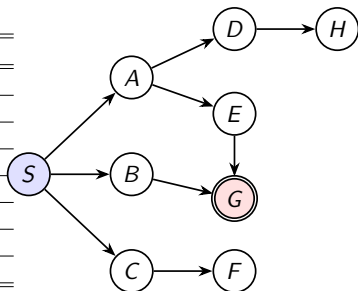


Selected: SA . Goal? No. Expand: add SAD, SAE .

Breadth-First Search: worked example

Reaching *G* from *S*:

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SB</i> not goal	{ <i>SC</i> , <i>SAD</i> , <i>SAE</i> , <i>SBG</i> }

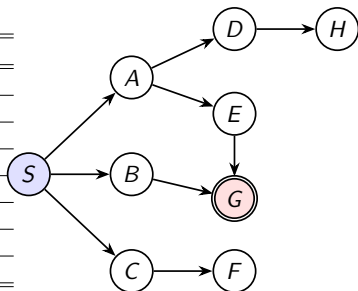


Selected: *SB*. Goal? No. Expand: add *SBG*.

Breadth-First Search: worked example

Reaching *G* from *S*:

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SB</i> not goal	{ <i>SC</i> , <i>SAD</i> , <i>SAE</i> , <i>SBG</i> }
<i>SC</i> not goal	{ <i>SAD</i> , <i>SAE</i> , <i>SBG</i> , <i>SCF</i> }

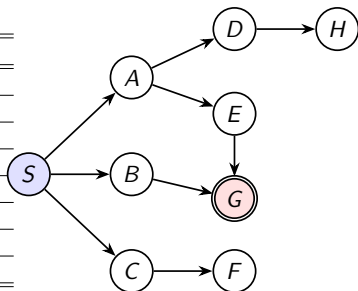


Selected: *SC*. Goal? No. Expand: add *SCF*.

Breadth-First Search: worked example

Reaching *G* from *S*:

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SB</i> not goal	{ <i>SC</i> , <i>SAD</i> , <i>SAE</i> , <i>SBG</i> }
<i>SC</i> not goal	{ <i>SAD</i> , <i>SAE</i> , <i>SBG</i> , <i>SCF</i> }
<i>SAD</i> not goal	{ <i>SAE</i> , <i>SBG</i> , <i>SCF</i> , <i>SADH</i> }

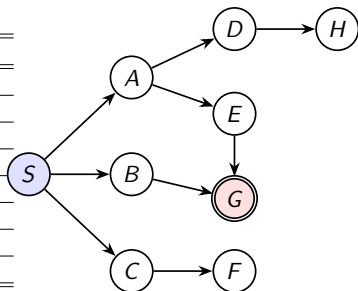


Selected: *SAD*. Goal? No. Expand: add *SADH*.

Breadth-First Search: worked example

Reaching *G* from *S*:

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SB</i> not goal	{ <i>SC</i> , <i>SAD</i> , <i>SAE</i> , <i>SBG</i> }
<i>SC</i> not goal	{ <i>SAD</i> , <i>SAE</i> , <i>SBG</i> , <i>SCF</i> }
<i>SAD</i> not goal	{ <i>SAE</i> , <i>SBG</i> , <i>SCF</i> , <i>SADH</i> }
<i>SAE</i> not goal	{ <i>SBG</i> , <i>SCF</i> , <i>SADH</i> , <i>SAEG</i> }

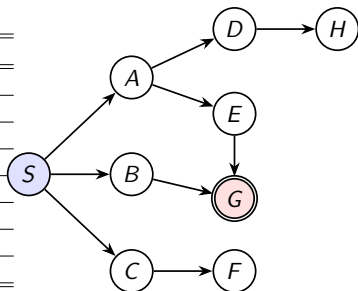


Selected: *SAE*. Goal? No. Expand: add *SAEG*.

Breadth-First Search: worked example

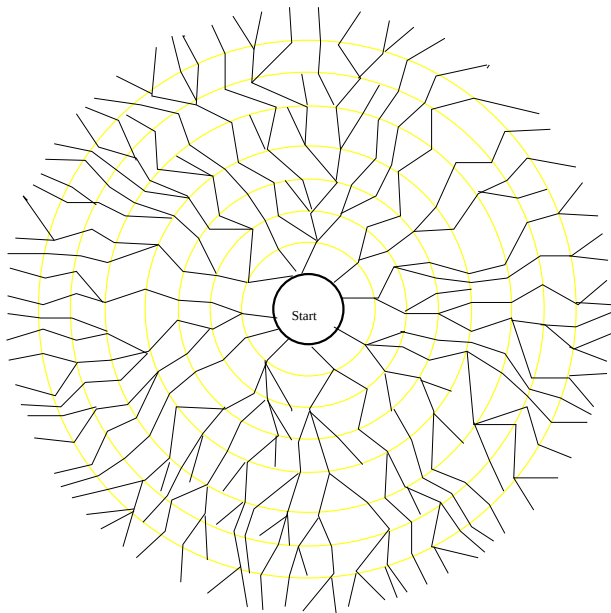
Reaching *G* from *S*:

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SB</i> not goal	{ <i>SC</i> , <i>SAD</i> , <i>SAE</i> , <i>SBG</i> }
<i>SC</i> not goal	{ <i>SAD</i> , <i>SAE</i> , <i>SBG</i> , <i>SCF</i> }
<i>SAD</i> not goal	{ <i>SAE</i> , <i>SBG</i> , <i>SCF</i> , <i>SADH</i> }
<i>SAE</i> not goal	{ <i>SBG</i> , <i>SCF</i> , <i>SADH</i> , <i>SAEG</i> }
<i>SBG</i> goal!	{ <i>SCF</i> , <i>SADH</i> , <i>SAEG</i> }



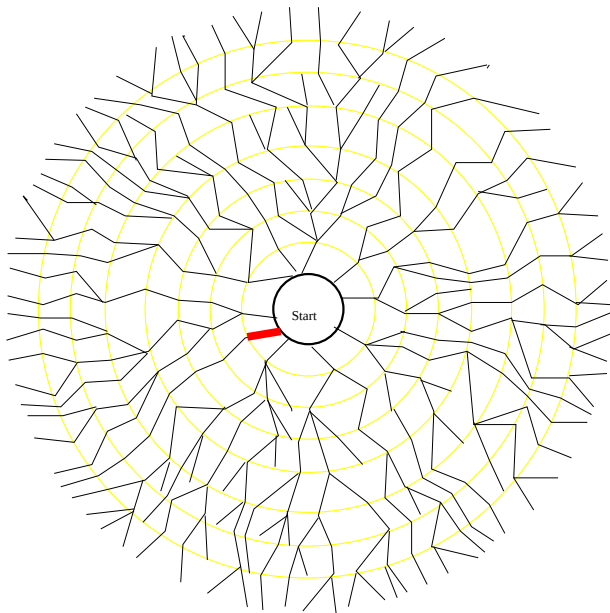
Selected: *SBG*. Goal? **Yes!** Solution: *SBG* — and note it is a **shortest** path to *G*.

Example: BFS in a Maze



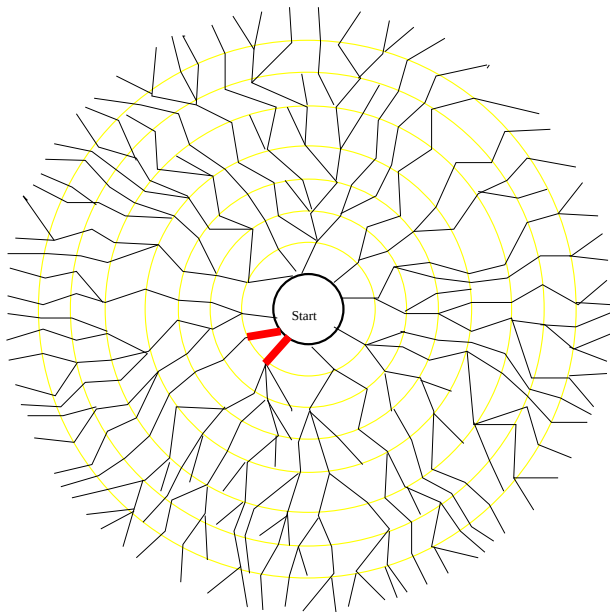
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



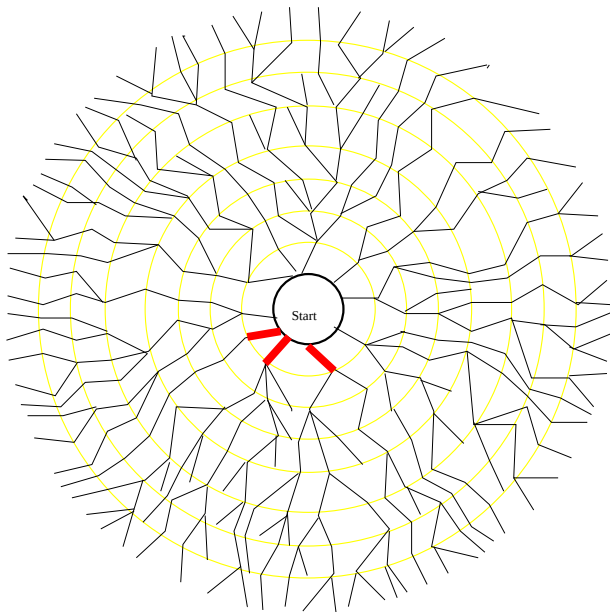
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



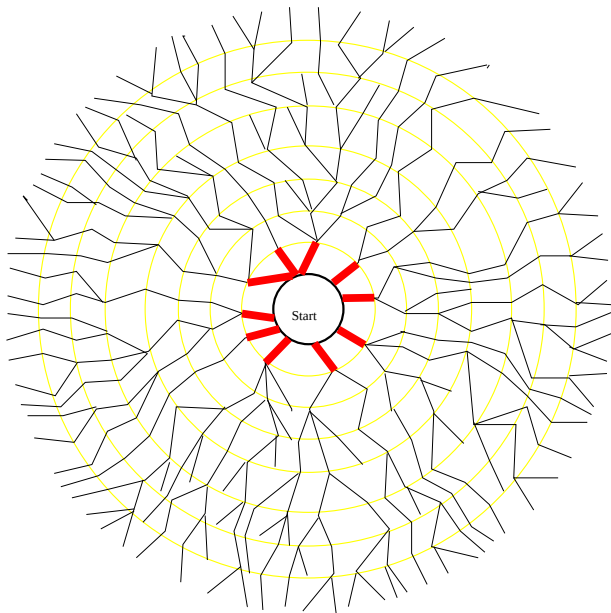
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



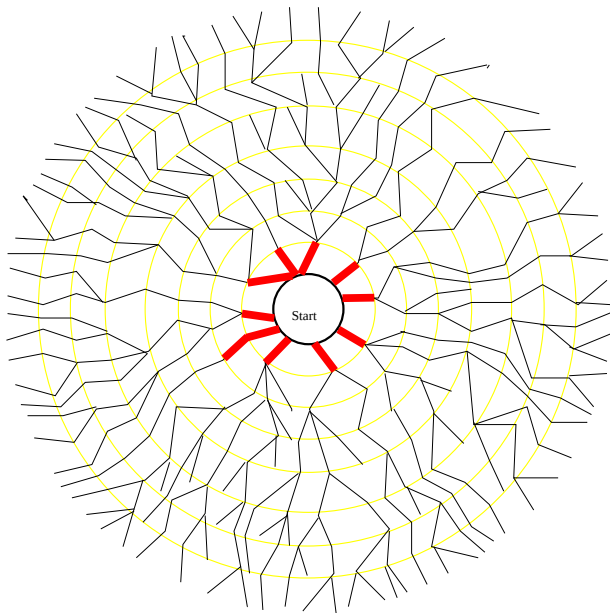
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



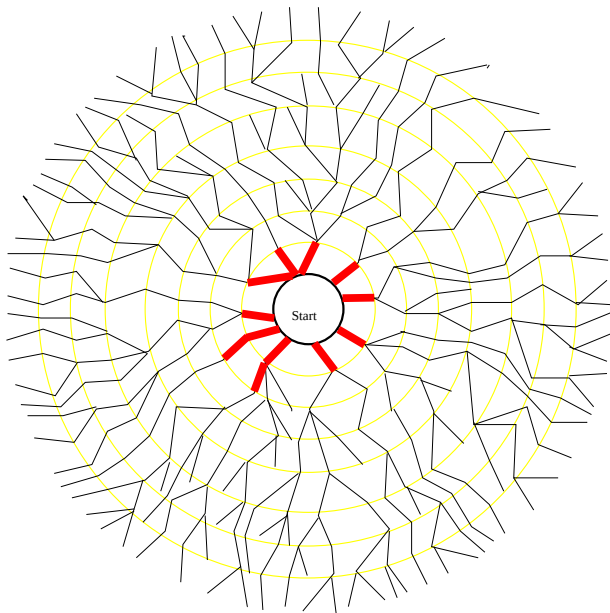
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



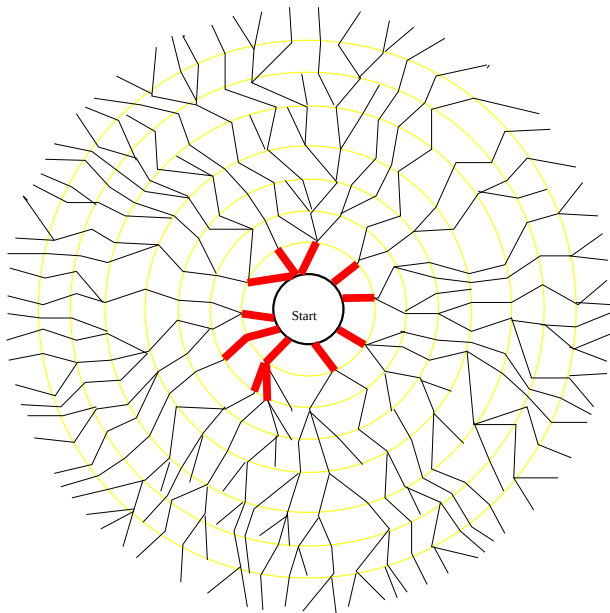
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



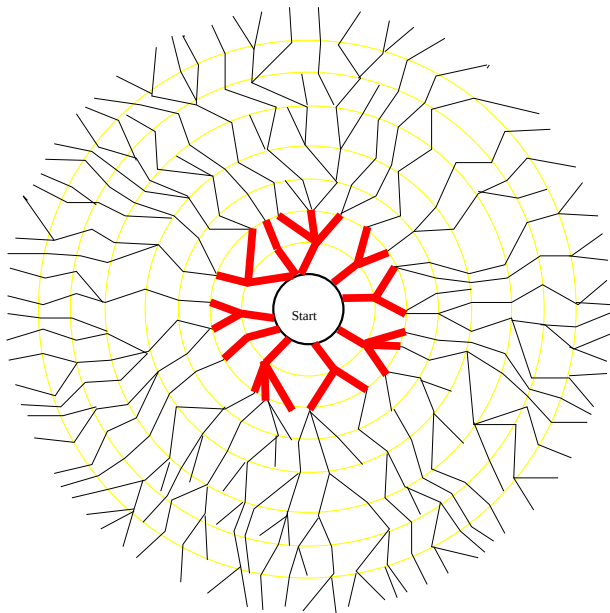
BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Example: BFS in a Maze



BFS spreads out evenly in all directions: it is **cautious** but **thorough**.

Depth-First Search (DFS)

- ▶ Start by expanding the start state;
- ▶ Select one of the resulting paths and expand it;
- ▶ Select one of *those* paths and expand it; and so on.

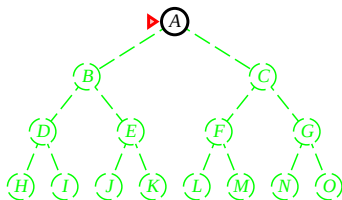
Depth-First Search (DFS)

- ▶ Start by expanding the start state;
- ▶ Select one of the resulting paths and expand it;
- ▶ Select one of *those* paths and expand it; and so on.
- ▶ Always select the **longest** (deepest) path: follow one branch of the tree to the end, then **backtrack**.

Depth-First Search (DFS)

- ▶ Start by expanding the start state;
- ▶ Select one of the resulting paths and expand it;
- ▶ Select one of *those* paths and expand it; and so on.
- ▶ Always select the **longest** (deepest) path: follow one branch of the tree to the end, then **backtrack**.

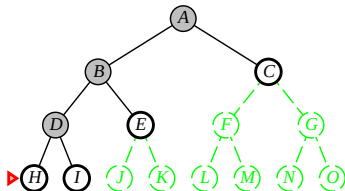
Implementation: the frontier is a **LIFO stack** — new paths go to the front.



Depth-First Search (DFS)

- ▶ Start by expanding the start state;
- ▶ Select one of the resulting paths and expand it;
- ▶ Select one of *those* paths and expand it; and so on.
- ▶ Always select the **longest** (deepest) path: follow one branch of the tree to the end, then **backtrack**.

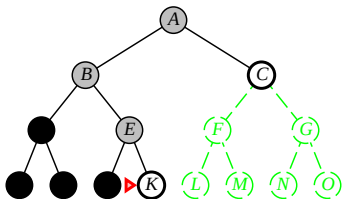
Implementation: the frontier is a **LIFO stack** — new paths go to the front.



Depth-First Search (DFS)

- ▶ Start by expanding the start state;
- ▶ Select one of the resulting paths and expand it;
- ▶ Select one of *those* paths and expand it; and so on.
- ▶ Always select the **longest** (deepest) path: follow one branch of the tree to the end, then **backtrack**.

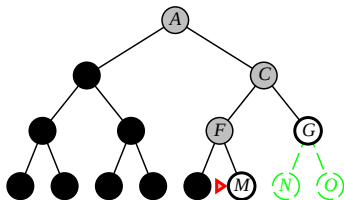
Implementation: the frontier is a **LIFO stack** — new paths go to the front.



Depth-First Search (DFS)

- ▶ Start by expanding the start state;
- ▶ Select one of the resulting paths and expand it;
- ▶ Select one of *those* paths and expand it; and so on.
- ▶ Always select the **longest** (deepest) path: follow one branch of the tree to the end, then **backtrack**.

Implementation: the frontier is a **LIFO stack** — new paths go to the front.



Depth-First Search: pseudocode

For the **select** step, always select the path **last added** to the frontier:

- 1: **Input:** a start state s_0
- 2: for each state s the successors of s
- 3: a test $\text{goal}(s)$ checking whether s is a goal state
- 4:
- 5: Set $\text{frontier} := \{s_0\}$
- 6: **while** frontier is not empty **do**
- 7: select and remove from frontier the path $s_0 \dots s_k$ that was
- 8: **last** added to frontier
- 9: **if** $\text{goal}(s_k)$ **then**
- 10: return $s_0 \dots s_k$ (and terminate)
- 11: **else** for every successor s of s_k add $s_0 \dots s_k s$ to frontier
- 12: **end if**
- 13: **end while**

Depth-First Search: pseudocode

For the **select** step, always select the path **last added** to the frontier:

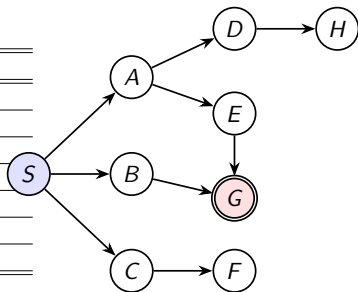
- 1: **Input:** a start state s_0
- 2: for each state s the successors of s
- 3: a test $\text{goal}(s)$ checking whether s is a goal state
- 4:
- 5: Set $\text{frontier} := \{s_0\}$
- 6: **while** frontier is not empty **do**
- 7: select and remove from frontier the path $s_0 \dots s_k$ that was
- 8: **last** added to frontier
- 9: **if** $\text{goal}(s_k)$ **then**
- 10: return $s_0 \dots s_k$ (and terminate)
- 11: **else** for every successor s of s_k add $s_0 \dots s_k s$ to frontier
- 12: **end if**
- 13: **end while**

Only one word has changed: “first” $\rightarrow$ “last”.

Depth-First Search: worked example

Reaching G from S (same graph):

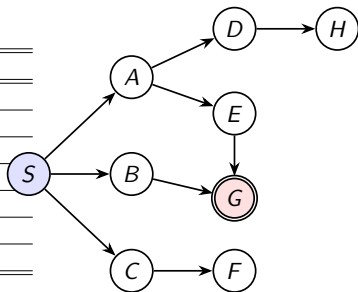
Expanded path	Frontier
	$\{S\}$



Depth-First Search: worked example

Reaching G from S (same graph):

Expanded path	Frontier
	$\{S\}$
S not goal	$\{SA, SB, SC\}$

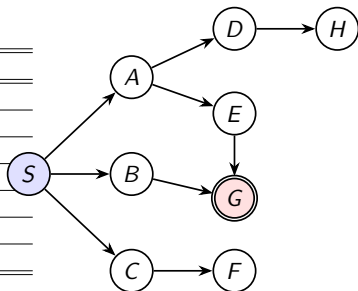


Selected: S . Goal? No. Expand: add SA, SB, SC .

Depth-First Search: worked example

Reaching G from S (same graph):

Expanded path	Frontier
	$\{S\}$
S not goal	$\{SA, SB, SC\}$
SA not goal	$\{SB, SC, SAD, SAE\}$

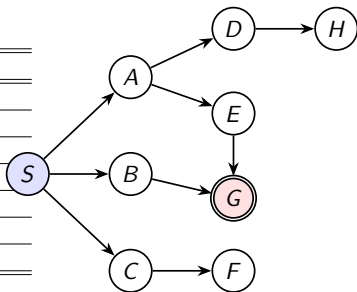


Selected: SA (last added). Goal? No. Expand: add SAD, SAE .

Depth-First Search: worked example

Reaching G from S (same graph):

Expanded path	Frontier
	$\{S\}$
S not goal	$\{SA, SB, SC\}$
SA not goal	$\{SB, SC, SAD, SAE\}$
SAD not goal	$\{SB, SC, SAE, SADH\}$

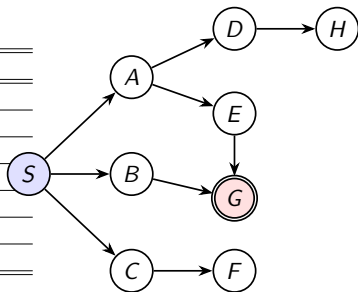


Selected: SAD . Goal? No. Expand: add $SADH$.

Depth-First Search: worked example

Reaching *G* from *S* (same graph):

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SAD</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAE</i> , <i>SADH</i> }
<i>SADH</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAE</i> }

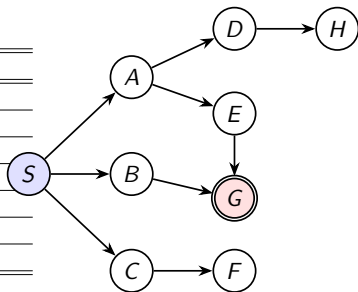


Selected: *SADH*. Goal? No. Nothing to add — **backtrack**.

Depth-First Search: worked example

Reaching *G* from *S* (same graph):

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SAD</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAE</i> , <i>SADH</i> }
<i>SADH</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAE</i> }
<i>SAE</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAEG</i> }

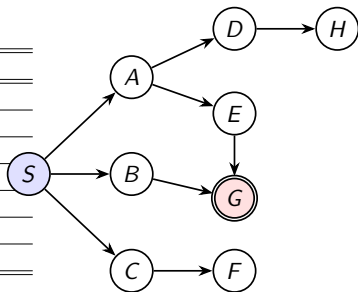


Selected: *SAE*. Goal? No. Expand: add *SAEG*.

Depth-First Search: worked example

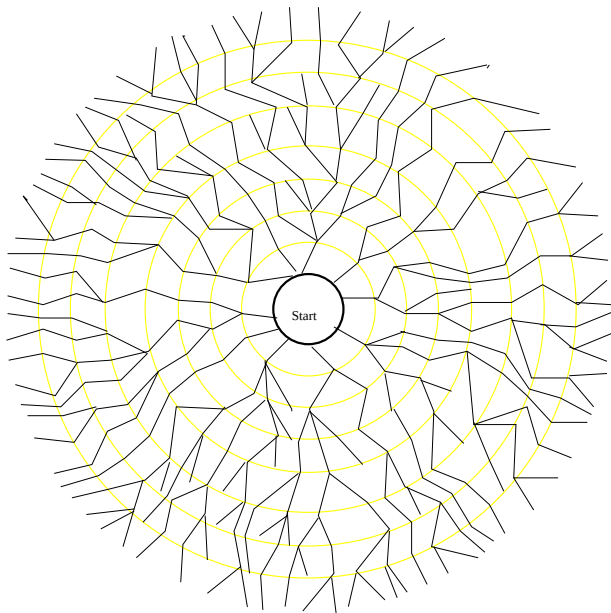
Reaching *G* from *S* (same graph):

Expanded path	Frontier
	{ <i>S</i> }
<i>S</i> not goal	{ <i>SA</i> , <i>SB</i> , <i>SC</i> }
<i>SA</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAD</i> , <i>SAE</i> }
<i>SAD</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAE</i> , <i>SADH</i> }
<i>SADH</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAE</i> }
<i>SAE</i> not goal	{ <i>SB</i> , <i>SC</i> , <i>SAEG</i> }
<i>SAEG</i> goal!	{ <i>SB</i> , <i>SC</i> }



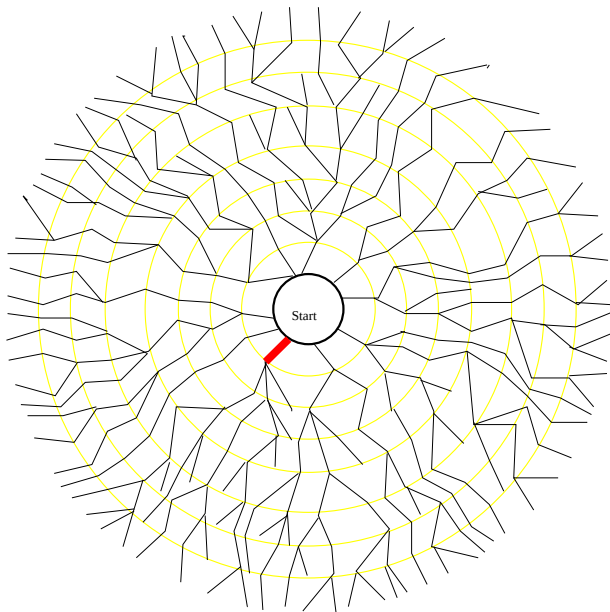
Selected: *SAEG*. Goal? **Yes!** Solution: *SAEG* — of length 3, whereas BFS found the shorter *SBG*. **DFS is not optimal.**

Example: DFS in a Maze



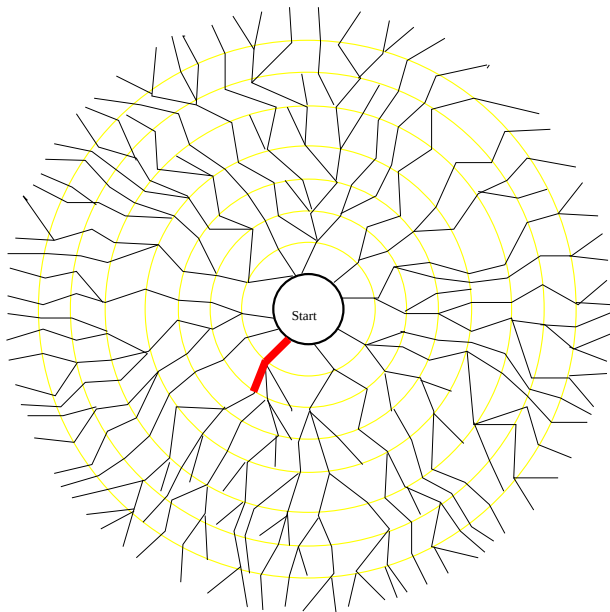
DFS commits to one direction and backtracks: it is **bold** but easily lost.

Example: DFS in a Maze



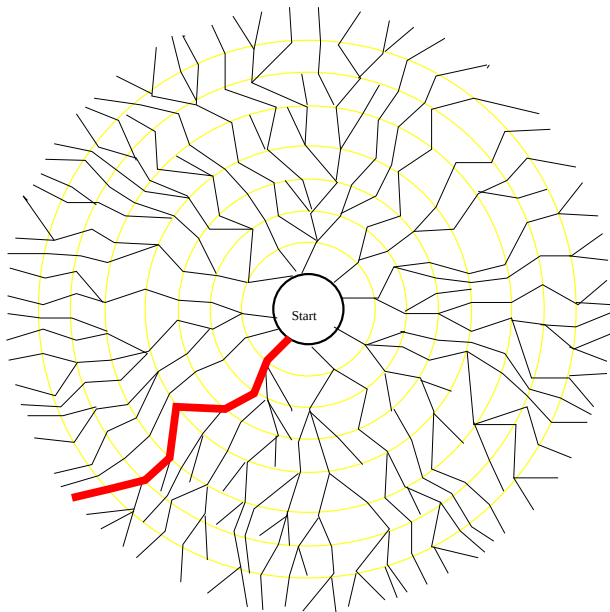
DFS commits to one direction and backtracks: it is **bold** but easily lost.

Example: DFS in a Maze



DFS commits to one direction and backtracks: it is **bold** but easily lost.

Example: DFS in a Maze



DFS commits to one direction and backtracks: it is **bold** but easily lost.

Properties of Search Strategies

- ▶ **Completeness:** does the algorithm always find a solution if one exists?

Properties of Search Strategies

- ▶ **Completeness:** does the algorithm always find a solution if one exists?

YES for BFS, NO for DFS (consider cycles in the search graph).

Properties of Search Strategies

- ▶ **Completeness:** does the algorithm always find a solution if one exists?

YES for BFS, NO for DFS (consider cycles in the search graph).

- ▶ **Optimality:** does the algorithm always find a shortest path (a path of lowest cost)?

Properties of Search Strategies

- ▶ **Completeness:** does the algorithm always find a solution if one exists?

YES for BFS, NO for DFS (consider cycles in the search graph).

- ▶ **Optimality:** does the algorithm always find a shortest path (a path of lowest cost)?

YES for BFS, NO for DFS.

Properties of Search Strategies

- ▶ **Completeness**: does the algorithm always find a solution if one exists?

YES for BFS, NO for DFS (consider cycles in the search graph).

- ▶ **Optimality**: does the algorithm always find a shortest path (a path of lowest cost)?

YES for BFS, NO for DFS.

- ▶ **Time complexity**: how many paths are generated?

Properties of Search Strategies

- ▶ **Completeness**: does the algorithm always find a solution if one exists?

YES for BFS, NO for DFS (consider cycles in the search graph).

- ▶ **Optimality**: does the algorithm always find a shortest path (a path of lowest cost)?

YES for BFS, NO for DFS.

- ▶ **Time complexity**: how many paths are generated?
- ▶ **Space complexity**: how many paths are held in memory (in the frontier) at once?

Properties of Search Strategies

- ▶ **Completeness**: does the algorithm always find a solution if one exists?

YES for BFS, NO for DFS (consider cycles in the search graph).

- ▶ **Optimality**: does the algorithm always find a shortest path (a path of lowest cost)?

YES for BFS, NO for DFS.

- ▶ **Time complexity**: how many paths are generated?
- ▶ **Space complexity**: how many paths are held in memory (in the frontier) at once?

For the last two we first need a way of measuring the size of the problem.

Measuring the Problem: b , d , m

Time and space complexity are measured in terms of:

- ▶ b — the **maximum branching factor** of the search tree: the largest number of successors any state has;

Measuring the Problem: b , d , m

Time and space complexity are measured in terms of:

- ▶ b — the **maximum branching factor** of the search tree: the largest number of successors any state has;
- ▶ d — the **depth of the optimal solution**: the length of the shortest path from s_{start} to a goal state;

Measuring the Problem: b , d , m

Time and space complexity are measured in terms of:

- ▶ b — the **maximum branching factor** of the search tree: the largest number of successors any state has;
- ▶ d — the **depth of the optimal solution**: the length of the shortest path from s_{start} to a goal state;
- ▶ m — the **maximum depth of the state space**: the length of the longest path in the search tree (may be ∞).

Measuring the Problem: b , d , m

Time and space complexity are measured in terms of:

- ▶ b — the **maximum branching factor** of the search tree: the largest number of successors any state has;
- ▶ d — the **depth of the optimal solution**: the length of the shortest path from s_{start} to a goal state;
- ▶ m — the **maximum depth of the state space**: the length of the longest path in the search tree (may be ∞).

Note $m = \infty$ whenever the search graph contains a **cycle**: our search tree simply grows paths, and nothing stops it from going round the cycle forever.

b , d and m for our Four Examples

Problem	b	d	m
8-Puzzle	4	up to 31	∞

b , d and m for our Four Examples

Problem	b	d	m
8-Puzzle	4	up to 31	∞
Holiday in Romania	4	3 (Arad $\rightarrow$ Bucharest)	∞

b , d and m for our Four Examples

Problem	b	d	m
8-Puzzle	4	up to 31	∞
Holiday in Romania	4	3 (Arad $\rightarrow$ Bucharest)	∞
Vacuum world	2	up to 3	∞

b , d and m for our Four Examples

Problem	b	d	m
8-Puzzle	4	up to 31	∞
Holiday in Romania	4	3 (Arad $\rightarrow$ Bucharest)	∞
Vacuum world	2	up to 3	∞
8 Queens	64	8	8

b , d and m for our Four Examples

Problem	b	d	m
8-Puzzle	4	up to 31	∞
Holiday in Romania	4	3 (Arad $\rightarrow$ Bucharest)	∞
Vacuum world	2	up to 3	∞
8 Queens	64	8	8

- 8-Puzzle: $b = 4$ when the blank is in the centre. The hardest start states need 31 moves, e.g.

876	8 6
41	547
253	231

b , d and m for our Four Examples

Problem	b	d	m
8-Puzzle	4	up to 31	∞
Holiday in Romania	4	3 (Arad $\rightarrow$ Bucharest)	∞
Vacuum world	2	up to 3	∞
8 Queens	64	8	8

- ▶ 8-Puzzle: $b = 4$ when the blank is in the centre. The hardest start states need 31 moves, e.g.

876	8 6
41	547
253	231

- ▶ 8 Queens: every solution path has length exactly 8, so $d = m = 8$ and there are no cycles — the only example here with a **finite** search tree.

Properties of Breadth-First Search

Fact. In a search tree with maximum branching factor b there are at most b^d paths of length d , and at most $1 + b + b^2 + \dots + b^d$ paths of length at most d .

Properties of Breadth-First Search

Fact. In a search tree with maximum branching factor b there are at most b^d paths of length d , and at most $1 + b + b^2 + \dots + b^d$ paths of length at most d .

- ▶ **Time:** if the shortest path to a goal has length d , in the worst case BFS generates

$$1 + b + b^2 + \dots + b^d$$

paths before reaching a goal (**exponential**).

Properties of Breadth-First Search

Fact. In a search tree with maximum branching factor b there are at most b^d paths of length d , and at most $1 + b + b^2 + \dots + b^d$ paths of length at most d .

- ▶ **Time:** if the shortest path to a goal has length d , in the worst case BFS generates

$$1 + b + b^2 + \dots + b^d$$

paths before reaching a goal (**exponential**).

- ▶ **Space:** in the worst case the frontier holds b^d paths (**exponential**).

Properties of Breadth-First Search

Fact. In a search tree with maximum branching factor b there are at most b^d paths of length d , and at most $1 + b + b^2 + \dots + b^d$ paths of length at most d .

- ▶ **Time:** if the shortest path to a goal has length d , in the worst case BFS generates

$$1 + b + b^2 + \dots + b^d$$

paths before reaching a goal (**exponential**).

- ▶ **Space:** in the worst case the frontier holds b^d paths (**exponential**).
- ▶ **Complete:** guaranteed to find a solution if one exists.

Properties of Breadth-First Search

Fact. In a search tree with maximum branching factor b there are at most b^d paths of length d , and at most $1 + b + b^2 + \dots + b^d$ paths of length at most d .

- ▶ **Time:** if the shortest path to a goal has length d , in the worst case BFS generates

$$1 + b + b^2 + \dots + b^d$$

paths before reaching a goal (**exponential**).

- ▶ **Space:** in the worst case the frontier holds b^d paths (**exponential**).
- ▶ **Complete:** guaranteed to find a solution if one exists.
- ▶ **Optimal:** always finds a shortest solution (when all actions cost the same).

Properties of Breadth-First Search

Fact. In a search tree with maximum branching factor b there are at most b^d paths of length d , and at most $1 + b + b^2 + \dots + b^d$ paths of length at most d .

- ▶ **Time:** if the shortest path to a goal has length d , in the worst case BFS generates

$$1 + b + b^2 + \dots + b^d$$

paths before reaching a goal (**exponential**).

- ▶ **Space:** in the worst case the frontier holds b^d paths (**exponential**).
- ▶ **Complete:** guaranteed to find a solution if one exists.
- ▶ **Optimal:** always finds a shortest solution (when all actions cost the same).

Space, not time, is what usually kills BFS in practice.

Combinatorial Explosion

Depth	Paths	Time
0	1	1 msec
1	11	.01 sec
2	111	.1 sec
4	11,111	11 secs
6	10^6	18 mins
8	10^8	31 hours
10	10^{10}	128 days
12	10^{12}	35 years

Combinatorial explosion!

Time for breadth-first search, assuming a branching factor of 10 and roughly 1000 paths expanded per second.

Properties of Depth-First Search

- ▶ **Not complete:** DFS may follow an infinite branch and never find an existing solution.

Properties of Depth-First Search

- ▶ **Not complete:** DFS may follow an infinite branch and never find an existing solution.
- ▶ **Not optimal:** the solution found need not be the shortest one (we saw *SAEG* vs *SBG*).

Properties of Depth-First Search

- ▶ **Not complete:** DFS may follow an infinite branch and never find an existing solution.
- ▶ **Not optimal:** the solution found need not be the shortest one (we saw *SAEG* vs *SBG*).
- ▶ **Time:** if the longest path from the start state has length m , in the worst case DFS generates

$$1 + b + b^2 + \dots + b^m$$

paths (**exponential**). If $m = \infty$, DFS may not terminate at all.

Properties of Depth-First Search

- ▶ **Not complete**: DFS may follow an infinite branch and never find an existing solution.
- ▶ **Not optimal**: the solution found need not be the shortest one (we saw *SAEG* vs *SBG*).
- ▶ **Time**: if the longest path from the start state has length m , in the worst case DFS generates

$$1 + b + b^2 + \dots + b^m$$

paths (**exponential**). If $m = \infty$, DFS may not terminate at all.

- ▶ **Space**: the frontier holds at most

$$b \times m$$

paths. If DFS does find a solution, its memory requirement is **far smaller** than that of BFS — linear rather than exponential in the depth.

Why is DFS space only $b \times m$? Watch the tree grow

depth 0

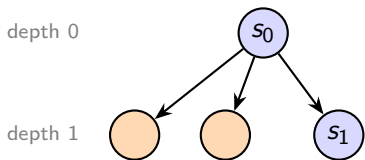


- the **current path**
- paths waiting **unexpanded** in the frontier (stubs)

Here $b = 3$, so each expansion leaves $b - 1 = 2$ paths behind.

Frontier = $\{s_0\}$, size 1.

Why is DFS space only $b \times m$? Watch the tree grow

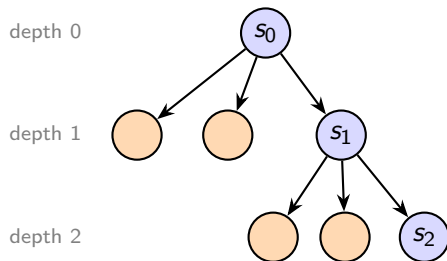


- the **current path**
- paths waiting **unexpanded** in the frontier (stubs)

Here $b = 3$, so each expansion leaves $b - 1 = 2$ paths behind.

Expand s_0 : add 3 children, dive down s_1 .
Size = 3.

Why is DFS space only $b \times m$? Watch the tree grow

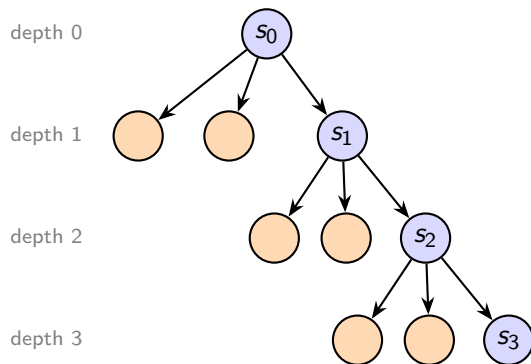


- the **current path**
- paths waiting **unexpanded** in the frontier (stubs)

Here $b = 3$, so each expansion leaves $b - 1 = 2$ paths behind.

Expand s_1 : 2 stubs stay at depth 1, add 3 more.
Size = $2 + 3 = 5$.

Why is DFS space only $b \times m$? Watch the tree grow



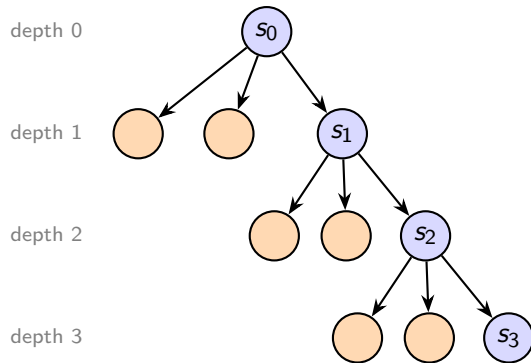
- the **current path**
- paths waiting **unexpanded** in the frontier (stubs)

Here $b = 3$, so each expansion leaves $b - 1 = 2$ paths behind.

Expand s_2 : another 2 stubs at depth 2.

Size = $2 + 2 + 3 = 7$.

Why is DFS space only $b \times m$? Watch the tree grow



- the **current path**
- paths waiting **unexpanded** in the frontier (stubs)

Here $b = 3$, so each expansion leaves $b - 1 = 2$ paths behind.

At depth k the frontier holds

$$k(b - 1) + 1$$

paths.

The bound $b \times m$, read off the pseudocode

```
1: frontier := {s0}
2: while frontier not empty do
3:   remove the last-added path
   s0 ... sk
4:   if goal(sk) then
5:     return s0 ... sk
6:   else
7:     push s0 ... sk s for each
       successor s
8:   end if
9: end while
```

One pass of the loop

- ▶ removes **one** path, and
- ▶ adds **at most b** (one per successor),
- ▶ then dives into the deepest.

So each level of the descent
leaves at most $b - 1$
un-expanded siblings behind.

The bound $b \times m$, read off the pseudocode

```
1: frontier := {s0}
2: while frontier not empty do
3:   remove the last-added path
   s0 ... sk
4:   if goal(sk) then
5:     return s0 ... sk
6:   else
7:     push s0 ... sk s for each
       successor s
8:   end if
9: end while
```

The current descent is a **single path**, so it has length at most m : it meets one node at each depth $0, 1, \dots, m$. Adding up the siblings it leaves behind,

$$|\text{frontier}| \leq \underbrace{(b-1)}_{\text{per level}} \times \underbrace{m}_{\text{levels}} + \underbrace{1}_{\text{current path}} \leq b \times m.$$

One pass of the loop

- removes **one** path, and
- adds **at most b** (one per successor),
- then dives into the deepest.

So each level of the descent leaves at most $b - 1$ un-expanded siblings behind.

The bound $b \times m$, read off the pseudocode

```
1: frontier := {s0}
2: while frontier not empty do
3:   remove the last-added path
   s0 ... sk
4:   if goal(sk) then
5:     return s0 ... sk
6:   else
7:     push s0 ... sk s for each
       successor s
8:   end if
9: end while
```

One pass of the loop

- ▶ removes **one** path, and
- ▶ adds **at most b** (one per successor),
- ▶ then dives into the deepest.

So each level of the descent
leaves at most $b - 1$
un-expanded siblings behind.

The current descent is a **single path**, so it has length at most m : it meets one node at each depth $0, 1, \dots, m$. Adding up the siblings it leaves behind,

$$|\text{frontier}| \leq \underbrace{(b-1)}_{\text{per level}} \times \underbrace{m}_{\text{levels}} + \underbrace{1}_{\text{current path}} \leq b \times m.$$

Linear in m — whereas BFS needs b^d paths at once.

BFS vs DFS: summary

	BFS	DFS
Frontier	queue (FIFO)	stack (LIFO)
Complete?	yes	no
Optimal?	yes	no
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$	$O(b \times m)$

BFS vs DFS: summary

	BFS	DFS
Frontier	queue (FIFO)	stack (LIFO)
Complete?	yes	no
Optimal?	yes	no
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$	$O(b \times m)$

- ▶ BFS is complete and optimal, but expensive — above all in **memory**;

BFS vs DFS: summary

	BFS	DFS
Frontier	queue (FIFO)	stack (LIFO)
Complete?	yes	no
Optimal?	yes	no
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$	$O(b \times m)$

- ▶ BFS is complete and optimal, but expensive — above all in **memory**;
- ▶ DFS is cheap in memory, but incomplete and non-optimal;

BFS vs DFS: summary

	BFS	DFS
Frontier	queue (FIFO)	stack (LIFO)
Complete?	yes	no
Optimal?	yes	no
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$	$O(b \times m)$

- ▶ BFS is complete and optimal, but expensive — above all in **memory**;
- ▶ DFS is cheap in memory, but incomplete and non-optimal;
- ▶ **Can't we do better than this?**

Note: Big O -Notation

A function g is in $O(f)$ if there exist a positive constant c and a threshold n_0 such that

$$|g(n)| \leq c \times f(n)$$

for all $n \geq n_0$. In computer science, O -notation lets us describe how an algorithm's running time or memory grows with the input size while ignoring hardware speed and implementation details. This matters most for scalability: two methods that look similar on small inputs can diverge enormously as the problem grows, which is exactly the combinatorial explosion.

Quick Quiz

1. In the vacuum world the search *graph* has only 8 states, yet $m = \infty$. Why?

Quick Quiz

1. In the vacuum world the search *graph* has only 8 states, yet $m = \infty$. Why?
2. A friend runs DFS on the Romania map and reports the route Arad, Sibiu, Arad, Sibiu, Arad, ... What has gone wrong, and what would you add to fix it?

Quick Quiz

1. In the vacuum world the search *graph* has only 8 states, yet $m = \infty$. Why?
2. A friend runs DFS on the Romania map and reports the route Arad, Sibiu, Arad, Sibiu, Arad, ... What has gone wrong, and what would you add to fix it?
3. For 8 Queens, does it matter whether we use BFS or DFS? Which would you choose, and why?

Quick Quiz

1. In the vacuum world the search *graph* has only 8 states, yet $m = \infty$. Why?
2. A friend runs DFS on the Romania map and reports the route Arad, Sibiu, Arad, Sibiu, Arad, ... What has gone wrong, and what would you add to fix it?
3. For 8 Queens, does it matter whether we use BFS or DFS? Which would you choose, and why?
4. You have a robot with 2MB of memory searching a graph with $b = 10$ and $d = 12$. Which algorithm do you run?

Try it Yourself

- ▶ Watch BFS and DFS race on a maze you draw yourself:
<https://qiao.github.io/PathFinding.js/visual/>

Try it Yourself

- ▶ Watch BFS and DFS race on a maze you draw yourself:
<https://qiao.github.io/PathFinding.js/visual/>
- ▶ Implement the generic algorithm on the 8-puzzle in 30 lines of Python: change one line (`frontier.pop(0)` → `frontier.pop()`) and BFS becomes DFS. Time both.

Try it Yourself

- ▶ Watch BFS and DFS race on a maze you draw yourself:
<https://qiao.github.io/PathFinding.js/visual/>
- ▶ Implement the generic algorithm on the 8-puzzle in 30 lines of Python: change one line (`frontier.pop(0)` → `frontier.pop()`) and BFS becomes DFS. Time both.
- ▶ Then ask yourself the question the next lecture answers:

Instead of expanding *any* path, why not expand the most *promising* one?

Next Lectures

Search strategies:

- ▶ **Blind** (uninformed) search: no information about states beyond the problem definition
 - ▶ breadth-first search ✓
 - ▶ depth-first search ✓
 - ▶ ...

Next Lectures

Search strategies:

- ▶ **Blind** (uninformed) search: no information about states beyond the problem definition
 - ▶ breadth-first search ✓
 - ▶ depth-first search ✓
 - ▶ ...
- ▶ **Heuristic** (informed) search: we can tell whether one non-goal state is “more promising” than another
 - ▶ greedy search
 - ▶ A* search